



Python Numpy & Pandas

By: Devesh Sir

What is Numpy?

NumPy (Numerical Python) is the fundamental open-source library for scientific computing in Python, providing support for large, multi-dimensional arrays and matrices.

Key Features of NumPy

Ndarray Object: The core is the ndarray, a fast, flexible, homogeneous, n -dimensional array object that enables efficient storage and manipulation of numerical data.

Performance: Written partially in C or C++, NumPy operations are significantly faster than standard Python lists, making it ideal for heavy computation.

Mathematical Functions: Includes tools for linear algebra, Fourier transforms, random number generation, and sophisticated sorting/selection.

Fixed-Size Arrays: Unlike Python lists, NumPy arrays have a fixed size at creation, optimizing memory usage and processing speed.

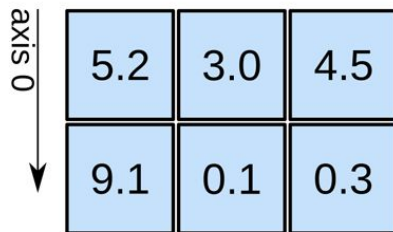
1D array



axis 0 →

shape: (4,)

2D array

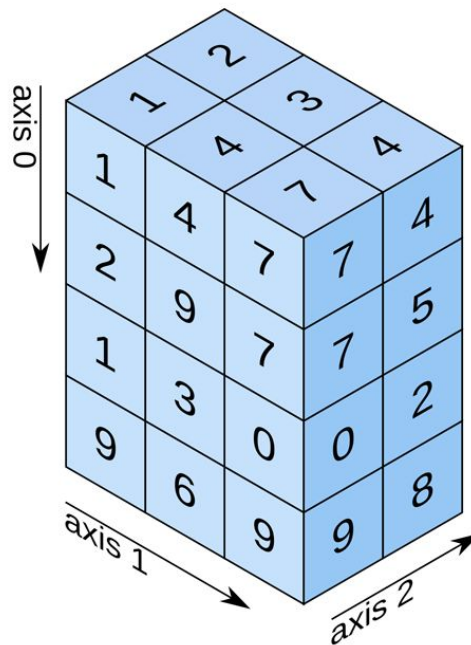


axis 0 ↓

axis 1 →

shape: (2, 3)

3D array

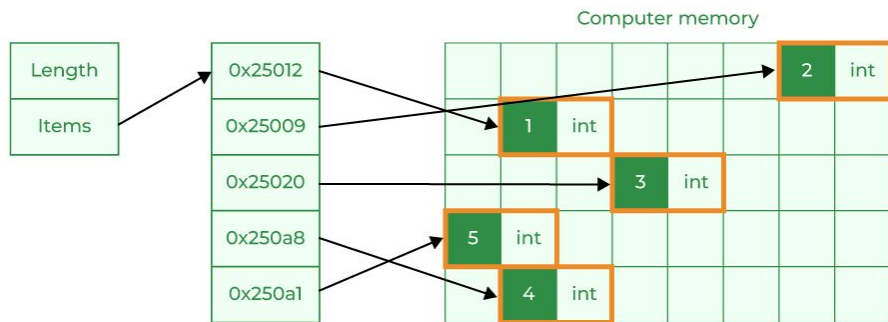


shape: (4, 3, 2)

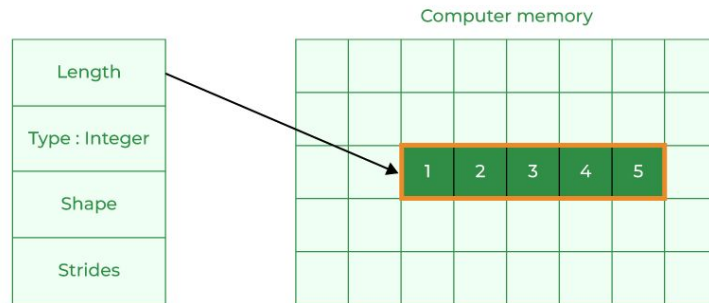
List Vs Numpy Array

Python lists are general-purpose, flexible data structures, while NumPy arrays are specialized for high-performance numerical computing and scientific tasks. The key differences lie in their performance, memory efficiency, and functionality.

Python List



NumPy Array



What is Numpy

- `Np.Array`
- `Np.Arrange` - Python Range Vs Arange
- Data type in array
- `NDarray` - 1D, 2D, 3D, ..
- `Arr.dtype`, `arr.size`, `arr.astype()`, `arr.shape`, `arr.ndim`
- `np.arange()`
- `np.reshape()`
- `Arr.Sum()`, `Arr.Mean()` - Aggregation In Arrays
- Indexing And Slicing
- Fancy Indexing In Numpy
- What Are Views In Numy
- What Is Broadcasting In Numpy
- `Np.Where`

Series

Series

A Pandas Series is a one-dimensional labeled data structure in Python, mainly used to store and work with a single column of data. You can think of it as an enhanced version of a list or NumPy array, but with labels (called an index) attached to each value.

```
import pandas as pd
```

```
s = pd.Series([230, 345, 120, 45, 670, 90])
```

```
s
```

```
0    230
```

```
1    345
```

```
2    120
```

```
3     45
```

```
4    670
```

```
5     90
```

```
dtype: int64
```


Attributes of Series

Attributes are properties of an object that give you information about it. They do not perform actions; instead, they describe the object. In Pandas, attributes help you understand the structure, size, data type, and labels of a Series.

- **index** – shows the labels (index) associated with each value in the Series.
- **values** – returns the actual data stored in the Series (as a NumPy array or array-like object).
- **dtype** – tells you the data type of the values, such as int64, float64, or object.
- **shape** – gives the dimensions of the Series (number of elements).
- **size** – returns the total number of elements in the Series.
- **ndim** – tells how many dimensions the Series has (always 1 for a Series).
- **name** – shows or sets the name of the Series.

Basic Aggregation Functions on Series

<code>s.sum()</code>	<code># → 150 → adds all values</code>
<code>s.mean()</code>	<code># → 30.0 → average value</code>
<code>s.median()</code>	<code># → 30.0 → middle value</code>
<code>s.mode()</code>	<code># → 10,20,... → most frequent value(s)</code>
<code>s.min()</code>	<code># → 10 → smallest value</code>
<code>s.max()</code>	<code># → 50 → largest value</code>
<code>s.count()</code>	<code># → 5 → number of non-null values</code>

np.where(logic test, true, false)

np.where() is a NumPy function used to apply conditional logic on arrays. It checks a condition and then chooses values based on whether the condition is True or False.

```
arr = np.array([10, 30, 60, 80])  
np.where(arr > 50, "High", "Low")
```

```
array(['Low', 'Low', 'High', 'High'])
```

For 1 condition

For checking more than one condition use

And - &

Or - |

```
np.where((df['wheel_base'] > 100) & (df['length'] > 175), "Buy", "Dont buy")
```

For 2 condition

.apply() method

.apply() method is used in pandas to apply a custom function to data without writing explicit loops. It allows you to run your own logic on each value, row, or column, which makes it very flexible, especially when built-in methods are not enough.

```
df["cost"] = df.apply(lambda row: row["Sales"] - row["Profit"], axis=1)
```

```
s = pd.Series([10, 20, 30, 40])  
s.apply(lambda x: x * x)
```

```
0    100  
1    400  
2    900  
3   1600  
dtype: int64
```

Single row

Multiple rows

Data Frame

DataFrame

A Pandas DataFrame is a two-dimensional data structure in Python used for storing and analyzing data. You can think of it like an Excel spreadsheet or a SQL table — it has rows and columns, where each column can contain data of different types (like numbers, text, or dates).

Column Label/ Header		0	1	2	3	4
Index Label		Name	Age	Marks	Grade	Hobby
0	S1	Joe	20	85.10	A	Swimming
1	S2	Nat	21	77.80	B	Reading
2	S3	Harry	19	91.54	A	Music
3	S4	Sam	20	88.78	A	Painting
4	S5	Monica	22	60.55	B	Dancing

Annotations:

- Column Index: Points to the column headers (0, 1, 2, 3, 4).
- Row Index: Points to the row indices (0, 1, 2, 3, 4).
- Column: Points to a specific column (e.g., Marks).
- Row: Points to a specific row (e.g., Row 3).
- Element/ Value/ Entry: Points to a specific cell value (e.g., 85.10).

load a file in Python using Pandas

Load a CSV file :

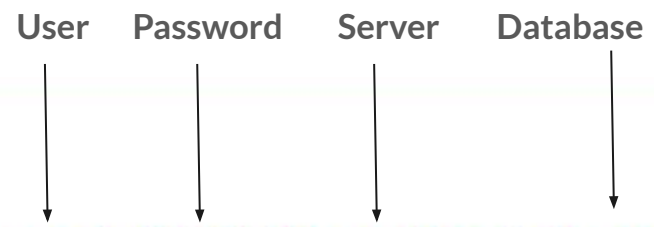
```
df = pd.read_csv('data.csv')
```

Load file from Excel

```
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
```

Getting Data From MySQL

```
import sqlalchemy as sql
engine = sql.create_engine(
    "mysql+mysqlconnector://root:1234@localhost/logistic"
)
df = pd.read_sql("select * from data;",engine)
```



The diagram illustrates the components of the MySQL connection string "mysql+mysqlconnector://root:1234@localhost/logistic" and their corresponding parts in the Python code. Arrows point from the labels "User", "Password", "Server", and "Database" to the respective parts of the connection string: "root" (User), "1234" (Password), "localhost" (Server), and "logistic" (Database).

if you don't have sqlalchemy install it

→ `pip install sqlalchemy`

Then run this command - `pip install mysql-connector-python`

How to look at a df

- **df.head()** - for top data by default 5
- **df.tail()** - for bottom data default 5
- **df.sample()** - for random data default 1

Check DataFrame and its content

1. **df.info()** - Gives column names, data types, non-null counts, and memory usage.
2. **df.shape** - Shape of DataFrame (1000, 12) means 1000 rows 12 columns.
3. **df.columns** - Lists all column names in the DataFrame.
4. **df.dtypes** - Shows the data type (int, float, object, datetime, etc.) for each column.
5. **df.describe()** - Gives count, mean, std deviation, min, max, and percentiles for numeric columns.
6. **df.isnull().sum()** - Shows how many null (missing) values are in each column.

Create new columns and delete columns

- Create a new column for profit percentage
 - `df['Profit'] = df['sales'] - df['cost']`
- You can remove a column using either of these methods:
 - `df = df.drop(columns='Profit')`
 - `del df['Region']`

Filter dataframe

- `df[["name", "sales", "profit"]]`

Filter DataFrame

1. Filter by one condition

```
# Age greater than 25
```

```
df[df['Age'] > 25]
```

```
# Age between 23 and 27 (inclusive)
```

```
df[df['Age'].between(23, 27)]
```

2. Filter by multiple conditions

```
# Age greater than 25 and City is 'Paris'
```

```
df[(df['Age'] > 25) & (df['City'] == 'Paris')]
```

```
# Age less than 25 or Score greater than 90
```

```
df[(df['Age'] < 25) | (df['Score'] > 90)]
```

Check Unique Values

```
df['column_name'].nunique()    # number of unique values  
df['column_name'].unique()    # list of unique values  
df['column_name'].value_counts() # frequency of each value
```

Handling Missing Data

- `df['col'].notna()` # for columns which don't have null values

```
df.isnull().sum()           # Check missing values
df.fillna(0, inplace=True)  # Replace missing with 0
df.dropna(inplace=True)     # Drop rows with missing data
```

Renaming Columns

```
df.rename(columns={'Name': 'StudentName'}, inplace=True)
```


loc and iloc

- `df.loc['CA-2016-155488',"sales"]`
- `df.loc["CA-2015-124891",["name","region","sales"]]`
- `df.iloc[0:21,:]`

Sort data

1. **Series** : `df['sales'].sort_values(ascending=False)`
2. **DataFrame** : `df.sort_values(by='Age', ascending=False)`
3. **Index** : `df.sort_index()`

Group By

1. Group by one column

```
df.groupby('City')['Sales'].sum()
```

2. Group by multiple columns

```
df.groupby(['City', 'Category'])['Sales'].sum()
```

3. Apply multiple aggregations

```
df.groupby('City')[['Sales', 'Profit']].agg(['sum', 'mean', 'max'])
```

```
df.groupby("Category").agg({"Sales":["sum","max"],"Profit":"mean"})
```

Index in DataFrame

```
# To get only index from the df  
df.index  
  
# This will add any column as index.  
df.set_index('Column', inplace=True)  
  
# This will remove the index from df  
df.reset_index(inplace=True)
```

Pivot Table

pivot table is used to summarize and analyze data just like in Excel. It lets you rearrange data by grouping and aggregating values easily.

```
df.pivot_table(  
    index="category",  
    columns='year',  
    values='profit',  
    aggfunc='sum'  
)
```

combine two DataFrames

- left → first DataFrame.
- right → second DataFrame.
- on → column(s) to match in both DataFrames.
- how → type of join ('inner', 'left', 'right', 'outer').
- You can merge using different column names using left_on and right_on

```
pd.merge(df1, df2, on='ID', how='inner')
```

pd.concat

```
pd.concat([emp, emp2])
```

Work with DateTime columns

```
# Convert a text column into datetime
df['OrderDate'] = pd.to_datetime(df['OrderDate'])

# Find and extract information from date
df['Year'] = df['OrderDate'].dt.year
df['Month'] = df['OrderDate'].dt.month
df['Day'] = df['OrderDate'].dt.day
df['order_date'].dt.month_name()
df['Weekday'] = df['OrderDate'].dt.day_name()

# Filter data
df[df['OrderDate'] >= '2025-11-02']
```


Exporting Data

```
df.to_csv('data.csv', index=False)  
df.to_excel('data.xlsx', index=False)
```

Turn Off Warnings In A Jupyter Notebook

If you are seeing a red warning before getting an output you can just turn it off . to turn off warnings in your code import warning module and call this function

```
import warnings
```

```
warnings.filterwarnings('ignore')
```